

Javascript Summery Chapter 1-4

Chapter 1

❖ **Web development:** is the art and science of building and maintaining of website and web applications.

✚ Types of web development:

- **Front-end Development(Client-Side):** involves creating the user interface and ensuring that the web content is visually appealing and interactive.
 - Front-end developers use HTML, CSS, and JavaScript
- **Back-end Development(Server-Side):** involves on the server-side of web applications.
 - Back-end developers work on server configuration, databases, and server-side.
 - Back-end developers use Node.js,python,Ruby, or PHP.

✚ Importance of Web Development:

- **Global Reach:** Websites and web apps are accessible worldwide with an internet connection.
- **Business and E-Commerce:** Supports businesses in reaching customers and enabling online transactions.
- **User Interaction:** Facilitates dynamic user engagement through interactivity (e.g., forms, responses).
- **Information Dissemination:** Acts as a primary medium for sharing news, education, and other resources.
- **Job Opportunities:** Offers career paths like front-end, back-end, full-stack development, and web design.

✚ Evolution of Web Development

- **HTML and Static Pages:** Initially, web pages were static and consisted mainly of HTML (Hypertext Markup Language).
- **Introduction of CSS:** CSS separated content (HTML) from design, enhancing visual presentation.

- **JavaScript Revolution:** Added interactivity and dynamic features, enabling real-time updates without page refreshes.
- **Rich Internet Applications (RIAs):** JavaScript enabled desktop-like interactivity in web applications.
- **Modern Web Development:** Today, web development involves complex frameworks, libraries, and tools.

❖ **Javascript - JS:** is a versatile and widely used programming language.

✚ Common Uses of JavaScript:

- **Building interactive web pages.**
- **Creating single-page applications (SPAs).**
- **Developing real-time apps (e.g., chat apps).**
- **Implementing animations and visual effects.**
- **Backend development using Node.js.**

✚ JavaScript enables you to:

- **Add Interactivity:** Create interactive elements like forms, animations, and real-time updates on web pages.
- **Respond to User Actions:** allows web pages to respond to user interactions like clicks, mouse movements, and keyboard input.
- **Validate and Manipulate Data:** Validate user input, perform calculations, and modify data on the client side.
- **Communicate with Servers:** Make asynchronous server requests to retrieve or send data and update pages dynamically.

✚ JavaScript is very different from another language called **Java**

✚ **Scripts** are provided plain text and do not require compilation.

History of JavaScript;

- **JavaScript** created by **Brendan Eich in 1995** while he was at Netscape. Originally named "**Mocha**" and later "**Livescript**" it was eventually named JavaScript.
- **Standardization (1996)**: began in collaboration with the European Computer Manufacturers Association (ECMA).
 - The first official specification, ECMAScript, was released in 1997 and continues to evolve.
- **Browser Wars (Late 1990s)**: between Netscape and Microsoft's Internet Explorer, led to JavaScript's rapid development as both browsers competed to improve JavaScript engines.
- **DOM and Ajax (Early 2000s)**: Document object model allowed dynamic web content manipulation.
 - **Ajax**: enabled asynchronous data exchange between browser And server.
- **JavaScript Frameworks (Mid-2000s)**: jQuery simplified DOM manipulation and ensured cross-browser compatibility.
- **HTML5 and Modern Web (Late 2000s - Early 2010s)**: JavaScript contributed to HTML5's native multimedia and advanced form features.
- **Node.js (2009)**: Brought JavaScript to server-side development, Unified language stack for scalable server applications.
- **ECMAScript 6 (2015)**: brought big improvements to JavaScript, making it easier and more powerful for developers to use. Like arrow function, classes, and template literals.

JavaScript's Role in Modern Web Development

- **Single-Page Applications (SPAs)**: dynamically update content without reloading the page.
- **Mobile Development**: JavaScript is used for building mobile app development using tools like React Native and Apache Cordova.

- **Responsive Design:** JavaScript helps web applications adapt to different devices and screen sizes.

+ JavaScript career opportunities are:

- **Front-End Developer:** specializes creating in user interfaces and interactive web elements using HTML, CSS and JavaScript.
- **Back-End Developer:** focus on server side development
- **Full-Stack Developer:** Masters both front-end and back-end development.

❖ **A text editor:** is your primary tool for writing and editing code. Popular text editors used by web developers are

+ **Visual Studio Code (VS Code):** a free open-source code editor developed by **Microsoft**. rich of ecosystem of extensions

+ **Sublime text:** a light weight and first text editor known for its speed and simplicity.

+ **Atom:** an open-source, hackable text editor created by **GitHub**.

+ **Brackets:** An open-source code editor designed for web development by **Adobe**.

❖ Browsers used for developments are

+ **Chrome**

+ **Mozilla firefox**

+ **Microsoft edge**

+ **Safari e.t.c**

Chapter 2

❖ Data are divided into two types

Primitive Data Types

- **Numbers:** represent numeric values, including both integers and floating-point numbers. For example

```
let age = 30;  
let price = 19.99;
```

- **Strings:** represent text or sequences of characters enclosed in single (') or double (") quotes. For example:

```
let name = "Mohamed";
```

- **Boolean (True or False):** represent binary value which can either be true or false and its used for making decisions and comparisons.

For example:

```
let isOpen = true;  
let isClose = false;
```

- **Undefined:** is a special value assigned to variables that have been declared but not yet assigned a value. For example

```
let undefinedVar;  
console.log(undefinedVar); // Outputs: undefined
```

- **Null:** represent the intentional absence of value or object and its used to indicate that a variable or property is empty or undefined.

For example:

```
let x = null; // x has no value
```

- **Symbol:** are unique and immutable values used as property keys in objects to prevent name collisions.

For example:

```
const id = Symbol('id');
```

✚ Non primitive Data Types

- **Objects:** containers storing key-value pairs.

For example:

```
const person = { name: "John", age: 30 };
```

- **Arrays:** list of items (elements) accessed by index.

For example:

```
const numbers = [1, 2, 3, 4, 5, 6];
```

- **Date:** a built-in object that handles dates and times.

For example:

```
const now = new Date();
```

❖ **Variables:** are used to store and manage data in JavaScript and it's also called **identifiers**.

✚ JavaScript variables can be declared in 4 ways

- Automatically

```
number = 10;
```

- Using **let**: **let** allows you to change the value of the variable.

```
let age = 20;
```

- Using **const**: **const** declares a variable whose value cannot be change after assignment.

```
const name = "bakar"
```

- Using **var**: **var** an older way to declare variables, with scoping issues, less recommended today.

```
var greeting = "Hello buddy"
```

❖ Difference between **Primitive Data Types** and **Non-Primitive Data Types**
(Composite Data Types)

✚ Variables of **primitive data types** hold the actual value of the data. (e.g., **Numbers** and **Strings**)

✚ Variables of **non-primitive data types** hold only reference value to the data. (e.g., **Objects** and **Arrays**)

NOTE: variables are containers that hold values, like cardboard box storing items, variables are not values themselves.

❖ Key rules for naming variables in JavaScript

- ✚ Variable names must start with a letter, Underscore (_), or Dollar sign (\$).
- ✚ Subsequent characters can include letters, digits, underscore, and dollar sign.
- ✚ Variable names are case-sensitive.
- ✚ Keywords or Reserved words cannot be used.
- ✚ Use descriptive and meaningful names.

❖ The full list of keywords and reserved words

break, case, catch, class, const, continue, debugger, default, delete, do, else, enum, export, extends, false, finally, for, function, if, implements, import, interface, in, instanceof, let, new, package, private, protected, public, return, static, super, switch, this, throw, true, try, typeof, var, void, while, with, yield.

❖ **JavaScript comments** are hints (notes) that a programmer can add to make their code easier to read and understand. Ignored (not displayed) by JS engine.

- ✚ There are two ways to add comments to code
 - // A single line comment
 - /* A multiple line comment */

❖ **Operators:** symbols that perform operations on values✚ **Arithmetic operator**

Operator	Description
+	Addition
-	Subtraction
*	Multiplication
**	Exponentiation
/	Division
%	Modulus (Remainder)
++	Increment
--	Decrement

Example:

```
let result = 5 + 3;
console.log(result) // result is 8
```

Comparison Operator

Operator	Description
==	equal to
===	equal value and equal type
!=	not equal
!==	not equal value or not equal type
>	greater than
<	less than
>=	greater than or equal to
<=	less than or equal to
?	ternary operator

Example:

```
let isEqual = 2 === 3;
console.log(isEqual) // isEqual is false
```

Logical Operator

Operator	Example	Same As
=	x = y	x = y
+=	x += y	x = x + y
-=	x -= y	x = x - y
*=	x *= y	x = x * y
/=	x /= y	x = x / y
%=	x %= y	x = x % y
**=	x **= y	x = x ** y

Example:

```
let count = 10;
count += 5; // count is now 15
```

Assignment Operator

Operator	Description
&&	logical and
	logical or
!	logical not

Example:

```
let isTrue = true && false;
console.log(isTrue) // false
```

```
let isTrue = true || false;
console.log(isTrue) // true
```

```
Pre let isTrue = !true;
console.log(isTrue) // false
```


Other operators

- **?:** Conditional (**ternary**) operator

Example:

```
let age = 20;
let message = age > 18 ? "Adult" : "Junior";
Console.log(message) // Adult
```

NOTE: Not all operators are symbols, some are written as words such as **typeof**.

- ❖ JavaScript has two unary operators: **increment (++)** and **decrement (--)**

Increment Operator (++)

Postfix Example:

```
let x = 5;
let y = x++; // y = 5, x = 6
```

Prefix Example:

```
let x = 10;
let y = ++x; // y = 11, x = 11
```

Decrement Operator (--)

Postfix Example:

```
let x = 8;
let y = x--; // y will be 8, and x will be 7
```

Prefix Example:

```
let x = 15;
let y = --x; // y will be 14, and x will be 14
```

❖ The **typeof** operator returns the type of a value as a string.

```
typeof "John"           // Returns "string"
typeof 3.14             // Returns "number"
typeof NaN              // Returns "number"
typeof false            // Returns "boolean"
typeof [1,2,3,4]        // Returns "object"
typeof {name:'John', age:34} // Returns "object"
typeof new Date()       // Returns "object"
typeof function () {}   // Returns "function"
typeof myCar             // Returns "undefined" *
typeof null             // Returns "object"
```

❖ **Automatic Type conversion** also known as **Type coercion**; converts values from one data type to another in certain situations.

 Common scenarios of automatic type conversion

String Concatenation implicit conversion

```
let age = 25;
let message = "I am " + age + " years old."; // Implicitly
converts age to a string
```

Numeric Operations implicit conversion

```
let numStr = "42";
let result = numStr * 2; // Implicitly converts numStr to a
number: result is 84
```

Comparison Operators implicit conversion

```
let num = 10;
let strNum = "5";
console.log(num > strNum);
```

// Implicitly converts strNum to a number for comparison: true

❖ Common JavaScript built-in functions and methods

Function/Method	Description
alert(message)	Displays a dialog box with a specified message.
prompt(message)	Displays a dialog box with a prompt for user input.
confirm(message)	Displays a dialog box with a confirmation message.
console.log(data)	Outputs data to the browser's console for debugging.
document.write(text)	Writes HTML content directly to the document.

❖ Escape characters

Escape Sequence	Character Represented	Description
<code>\\n</code>	Newline	Inserts a newline character.
<code>\\t</code>	Tab	Inserts a tab character.
<code>\'</code>	Single Quote	Escapes a single quote within a string.
<code>\"</code>	Double Quote	Escapes a double quote within a string.
<code>\\</code>	Backslash	Escapes a backslash within a string.

Chapter 3

❖ **Template literals** or **Template String**: allows you to embed expressions and variables within a string; and makes it easy to create dynamic content.

🚦 Template String are enclosed in backticks (``...``)

```
const name = "Alice";
const message = `Hello, ${name}!`;
```

✚ Advantage of Template literals

- String interpolation: variable iyo string inuu isdhax galiyo.
- Multiline strings: you can write multiline strings using one template literal

❖ **Variable Swapping:** is the process of exchanging values between two variables.

✚ There are various methods to swap two variables

- **Using a Temporary Variables**



```

1 let a = 5;
2 let b = 10;
3
4 a = a + b;
5 b = a - b;
6 a = a - b;
7
8 console.log("After swapping: a = " + a + ", b = " + b);
```

- **Using Arithmetic Operations**



```

1 let a = 5;
2 let b = 10;
3
4 a = a + b;
5 b = a - b;
6 a = a - b;
7
8 console.log("After swapping: a = " + a + ", b = " + b);
```

❖ **Conditional Statement:** in JavaScript are used to make decisions in you code and allows you to execute different blocks of code based on whether it's true or false.

✚ Types of conditional statement are

- **if statement:** executes if the condition is true. and it's called **One way statement**.



```

1  let age = 18;
2
3  if (age >= 18) {
4    console.log("You are an adult.");
5  }

```

- **else statement:** executes if the condition is false. and it's called **Two-way statement**.



```

1  let age = 18;
2
3  if (age >= 18) {
4    console.log("You are an adult.");
5  } else {
6    console.log("You are a minor.");
7  }

```

- **else-if statement:** checks a new condition if the previous one is false. And it's called **Multi-way statement or Multi-conditional statement**.



```

1  let grade = 85;
2
3  if (grade >= 90) {
4    console.log("A");
5  } else if (grade >= 80) {
6    console.log("B");
7  } else {
8    console.log("C");
9  }

```

❖ **Ternary Operator (? :):** provides a concise way to express conditional statement in a single line. It's used when you want to assign value based on a condition.



```
1 let isAdult = age >= 18 ? true : false;
```

❖ **Switch statement:** is used to perform different actions based on different conditions. It works as follows

- ✚ The switch expression is evaluated once.
- ✚ The value of the expression is compared with the values of each case.
- ✚ If there is a match, the associated block of code is executed.
- ✚ If there is no match, the default code block is executed.



```
1 let day = "Saturday";
2
3 switch (day) {
4   case "Saturday":
5     console.log("It's the start of the week.");
6     break;
7   case "Wednesday":
8     console.log("It's almost the weekend!");
9     break;
10  default:
11    console.log("It's a regular day.");
12 }
```

❖ **Loops:** are used to execute a block of code repeatedly .

- ✚ The five primary loop types are:
 - **for Loop:** loops a specific numbers of items or over an array. and it consist of three parts
 - **Initialization:** this is where you initialize a loop control variable.
 - **Condition:** determines if the loop continues (true) or stop (false)

- **Increment/decrement:** updates the loop variable after each iteration.

- This is the syntax of a for loop

```
for (initialization; condition; increment/decrement) {  
  // code to be executed in each iteration  
}
```



```
1 for (let i = 1; i <= 5; i++) {  
2   console.log(i);  
3 }
```

- **while Loop:** loops as long as a condition is true. It checks the condition before each iteration.

- **Condition:** is the condition that's checked before each iteration; if true, the loop runs; if false, it stops.
- The syntax of a while loop

```
while (condition) {  
  // code to be executed as long as the condition is true  
}
```



```
1 let i = 1;  
2 while (i <= 5) {  
3   console.log(i);  
4   i++;  
5 }
```

- **do...while Loop:** similar to while loop, and executes at least once, then check the condition.
 - The syntax of do...while loop


```
do {
    // code to be executed
} while (condition);
```

NOTE: the do block runs first, then the condition is checked. The loop continues if true and stops if false.



```
1 let i = 1;
2 do {
3   console.log(i);
4   i++;
5 } while (i <= 5);
```

- **for...in Loop:** loops through the properties of an object.
- **for...of Loop:** loops through iterable objects like arrays, strings, maps and sets. And provides a simple and concise way to loop through values.

❖ Difference between Break and Continue statement

 **Break:** is used to exit or terminate a loop prematurely. Used when a condition is met.

```
for (let i = 0; i < 10; i++) {
    if (i === 5) break; // Stops the loop when i is 5
}
```

 **Continue:** skips the current iteration and moves to the next one.

```
for (let i = 0; i < 10; i++) {
    if (i === 5) continue; // Skips iteration when i is 5 }
}
```


- ❖ **Nested loop:** are loops inside another loop. Useful for handling multiple level of iteration, such as multi-dimensional arrays or generating combinations.



```

1 // Outer loop
2 for (let i = 0; i < 3; i++) {
3   // Inner loop
4   for (let j = 0; j < 3; j++) {
5     console.log(`${i}, ${j}`);
6   }
7 }

```

Chapter 4

- ❖ **JavaScript Functions:** are reusable block of code that perform a specific tasks. They can accept inputs (parameters).

- ✚ Functions help organize and reuse code, making you programs more modular and easier to manage.
- ✚ **JavaScript functions:** is a block of code designed to perform a particular task. its declared by using **function** keyword.



```

1 function functionName(parameters) {
2   // Function code here
3   // You can perform operations and return values
4 }

```

- ❖ Function structure/components

- ✚ **function:** this keyword used to declare functions in JS.
- ✚ **functionName:** the function's used to call it (follows variable naming rules)

 **parameters:** Optional values inside parentheses, passed when calling the function and separated by commas.

 **{}**: curly braces encloses the function's code block defines what the function does.

- Function without parameter and calling function by using it's name

```

1 function functionName() {
2     console.log("Hello, world!");
3 }
4 functionName(); // This will log "Hello, world!" to the console.

```

- Function declaration are hoisted, which means you can call a function before you declare it.

```

1 functionName(); // This will log "Hello, world!" to the console.
2
3 function functionName() {
4     console.log("Hello, world!");
5 }

```

❖ **Parameters:** are placeholders in a function's definition and can accept input values.

 They act as local variables inside the function, allowing you to pass information for the function to use.

```

1 function functionName(parameter1, parameter2, ...) {
2     // Function code using the parameters
3 }

```

❖ **Arguments:** are the actual values that you provide when calling a function.



```
1 functionName(argument1, argument2, ...);
2
```

NOTE: the arguments must match the number and order of parameters in the function declaration



```
1 function greet(name) {
2   console.log(`Hello, ${name}!`);
3 }
4
5 // "Mohamed" is an argument, and it will be assigned to the parameter "name"
6 greet("Mohamed");
```

❖ In JavaScript **return values** are the results a functions sends back to the calling code using **return** statement.



```
1 function functionName(parameters) {
2   // Function code here
3
4   return value; // This is the return statement
5 }
```

❖ **Variable Scope:** in JS defines where variables can be accessed in your code. There are two main types.



Global Scope

- Variables declared outside of any function and considered to be in global scope.
- Global variables can be accessed and modified from anywhere in your code including inside function.

- Variables declared with `var` in the global scope become properties of the global object.



```

1 var globalVariable = "I'm in global scope";
2
3 function sayHello() {
4   console.log(globalVariable); // Accessible from within the function
5 }
6
7 sayHello();
8 console.log(globalVariable); // Accessible outside the function

```

Local (function) scope

- Variables declared inside a function are in local scope.
- Local variable can only be accessed and modified within the function in which they are declared.
- Each function creates its own shape, and the variables with the same name in different functions do not interfere with each other.

❖ Block scope

-  Variables declared with `let` and `const` have block scope
-  It's limited to the block (e.g., loops or conditionals) where the variable is declared.
-  Variables with block scope cannot be accessed outside of their defining block.



```

1 if (true) {
2   let blockVariable = "I'm in block scope";
3   console.log(blockVariable); // Accessible within the block
4 }
5
6 console.log(blockVariable); // Results in an error; not accessible outside the block

```

- ❖ **Function parameter:** are in local scope of the function and accessible only within it. they act as local variables.

NOTE: Understanding **variable scope** helps prevent overwrites, naming conflicts, and hoisting issues. Using `let` and `const` for block scope ensures better control and avoids common mistakes in modern JavaScript.

❖ **Arrow functions:** in JS offer a concise and expressive way to define functions with simpler syntax.

✚ Format syntax is

`(parameter1, parameter2, ..., parameterN) => expression`

✚ The parameters represent the parameters that the function accepts. You can have zero, one or multiple parameters.

✚ The arrow `(=>)`: signifies that you're defining an arrow function.

✚ **Expressions:** is a single expression or statement that the function will execute.



```
1 const sayHello = () => "Hello, world!";
2 console.log(sayHello()); // "Hello, world!"sayHello
```



```
1 const doubleNumber = (number) => {
2   return number * 2;
3 }
4
5 console.log(doubleNumber(3)); // 6
```



```
1 const add = (a, b) => a + b;
2 console.log(add(3, 5)); // 8
```

❖ **Callback functions:** in JavaScript are functions passed as arguments to other functions and executed later, typically when certain events or conditions are met.

✚ How call works

- **Passing a function as an argument**
- **Execution at later time**

```

1  function doSomething(callback) {
2    console.log("Doing something...");
3    callback(); // Execute the callback function
4  }
5
6  function onComplete() {
7    console.log("Operation complete!");
8  }
9
10 doSomething(onComplete);

```

❖ Uses of Arrow function

- ✚ **Asynchronous Operations**
- ✚ **Event Handling**
- ✚ **Error Handling**
- ✚ **Iterations and Higher-Order Functions**

❖ **Anonymous Callback functions:** can be defined directly within the function call (as anonymous functions) or passed as references to named functions.

```

1  setTimeout(function () {
2    console.log("This is a callback executed after 2 seconds.");
3  }, 2000);

```

❖ **Callback function:** in JavaScript are essential for handling asynchronous tasks, events, and customizing function behavior.

- ✚ They allow functions to be passed as arguments, increasing code flexibility and reusability.

